

Introduction à Next.js : Routage, pages dynamiques et data fetching

Objectifs de cette section

Next.js est un framework basé sur React qui simplifie la création d'applications performantes, bien structurées et prêtes pour la production.

Dans cette section, vous allez découvrir les **fondamentaux de Next.js**.

À la fin de cette section, vous serez capable de :

- Comprendre le fonctionnement du **routage basé sur les fichiers**.
 - Créer des **pages statiques et dynamiques**.
 - Récupérer des données côté serveur ou client avec `getStaticProps`, `getServerSideProps`, ou `fetch`.
-

1. Le routage dans Next.js

Next.js utilise un système de **routage automatique** basé sur le contenu du dossier `/pages`.

- `pages/index.jsx` → `/`
- `pages/about.jsx` → `/about`
- `pages/products/[id].jsx` → `/products/42`

Pas besoin de déclarer vos routes dans un routeur !

Exemple : page simple

```
// pages/about.jsx
export default function About() {
  return <h1>À propos de notre site</h1>;
}
```

2. Pages dynamiques avec les paramètres d'URL

Vous pouvez créer une page dynamique avec la syntaxe `[param].jsx`.

Next.js injecte les paramètres via la fonction `getStaticPaths` (en SSG) ou via le contexte en SSR.

Exemple : page dynamique avec `[id]`

```
// pages/products/[id].jsx
import {useRouter} from "next/router";

export default function ProductPage() {
  const router = useRouter();
  const {id} = router.query;

  return <h1>Produit #{id}</h1>;
}
```

3. Récupération de données

Next.js permet de charger des données à plusieurs moments du cycle de vie :

- `getStaticProps` → au build (pages statiques)
- `getServerSideProps` → à chaque requête (pages dynamiques SSR)
- `fetch` ou `useEffect` → côté client (après le rendu)

Exemple : chargement avec `getStaticProps`

```
// pages/index.jsx
export async function getStaticProps() {
  const products = [
    {id: "1", name: "Produit A"},
    {id: "2", name: "Produit B"},
  ];

  return {
    props: {products},
  };
}

export default function Home({products}) {
  return (
    <ul>
      {products.map((p) => (
        <li key={p.id}>{p.name}</li>
      ))}
    </ul>
  );
}
```

4. Combiner données et routes dynamiques

On peut générer une liste de pages dynamiques statiques à partir de données via :

- `getStaticPaths` : liste des routes dynamiques à générer
- `getStaticProps` : contenu pour chaque page

Exemple : blog avec pages générées à partir d'un tableau d'articles

```
// pages/products/[id].jsx
export async function getStaticPaths() {
  const products = [
    {id: "1"},
    {id: "2"},
  ];

  const paths = products.map((p) => ({
    params: {id: p.id},
  }));

  return {paths, fallback: false};
}

export async function getStaticProps({params}) {
  const data = {id: params.id, name: `Produit ${params.id}`};

  return {
    props: {product: data},
  };
}

export default function ProductPage({product}) {
  return <h1>{product.name}</h1>;
}
```

5. Exercice de mise en pratique

Consignes

1. Créez un projet Next.js avec :
 - Une page `/` qui affiche une liste de produits.
 - Une page `/products/[id]` qui affiche les détails d'un produit.
 - Utilisez `getStaticProps` pour charger la liste.
 - Utilisez `getStaticPaths` + `getStaticProps` pour les pages produit.
2. Bonus : ajoutez une page `/about` statique.

Organisation attendue

```
/pages
├── index.jsx           # Liste de produits (static props)
├── about.jsx          # Page statique
└── products
    └── [id].jsx        # Page dynamique produit (static paths + props)

/public                # Images, icônes, etc.
```